

Reviewer Pack — Free Entropy Distinguishes Read-Twice BPs from IP_2 Trivial ...

Entry eea147175457 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-10 09:46:20 UTC

Free Entropy Distinguishes Read-Twice BPs from IP_2 Trivial Ones

Entry ID: eea147175457 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-10 09:46:20 UTC

1. Conjecture

Field A (mathematical branch): Free Probability **Field B** (complexity object): Read-Twice Branching Programs

Statement:

For a read-twice branching program P over n variables, let M_P be the matrix whose entries are the free entropy of the adjacency matrix of P 's layers. Then, the minimum eigenvalue of M_P is $\Theta(\log \text{size}(P))$, while for the IP_2 trivial BP, it is $\Omega(n)$.

Rationale (proposer's reasoning):

Free entropy captures spectral properties of random matrices, which may reveal structural differences between read-twice BPs (with logarithmic size) and the IP_2 trivial BP (with linear size). The tensor product structure of M_P ensures computable invariants.

Taxonomy category: BARRINGTON_ALG (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): aa8863f355b9ad77

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.95	SAFE	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_read_twice_bp(n):
        layers = []
        for _ in range(2): # Read-twice BP has two layers
            layer = [[random.randint(0, 1) for _ in range(n)] for _ in range(n)]
            layers.append(layer)
        return layers

    def adjacency_matrix(bp_layer):
        n = len(bp_layer)
        adj_mat = [[0] * n for _ in range(n)]
        for i in range(n):
            for j in range(n):
                if bp_layer[i][j]:
                    adj_mat[i][j] = 1
        return adj_mat

    def tensor_product(matrices):
        if not matrices:
            return []
        result = matrices[0]
        for mat in matrices[1:]:
            new_result = []
            for row1 in result:
                new_row = [sum(row1[j] * row2[j] for j in range(len(row2))) for row2 in mat]
                new_result.append(new_row)
            result = new_result
        return result

    def min_eigenvalue(matrix):
        n = len(matrix)
        eigenvalues = []
        for i in range(n):
            vector = [1 if j == i else 0 for j in range(n)]
            value = sum(matrix[i][j] * vector[j] for j in range(n))
```

```

        eigenvalues.append(value)
    return min(eigenvalues)

n = random.randint(5, 40)
bp = generate_read_twice_bp(n)
adj_matrices = [adjacency_matrix(layer) for layer in bp]
mp = tensor_product(adj_matrices)
min_eig = min_eigenvalue(mp)

size_p = sum(sum(row) for row in bp[0]) + sum(sum(row) for row in bp[1])
expected_min_eig = math.log(size_p)

if "IP_2 trivial BP" in str(bp):
    conjecture_holds = min_eig >= 0.9 * n
    counterexample = ""
else:
    conjecture_holds = abs(min_eig - expected_min_eig) <= 1e-5
    counterexample = "" if conjecture_holds else "IP_2 trivial BP"

return {
    "metric_name": "min_eigenvalue",
    "metric_value": min_eig,
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(arg) for arg in sys.argv[1:]] or [
        2, 3, 5, 7, 11, 13, 17, 19, 23, 29,
        31, 37, 41, 43, 47, 53, 59, 61, 67, 71,
        73, 79, 83, 89, 97, 101, 103, 107, 109, 113
    ]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(res["metric_value"] for res in results) / len(results)
    std_value = math.sqrt(sum((res["metric_value"] - mean_value) ** 2 for res in results) / len(results))
    support_fraction = sum(1 for res in results if res["conjecture_holds"]) / len(results)

    if all(res["conjecture_holds"] for res in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(res["seed"] for res in results if not res["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"IP_2 trivial BP\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

alue': 1, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': 'IP_2 trivial BP'}
TRIAL: {'metric_name': 'min_eigenvalue', 'metric_value': 1, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': 'IP_2 trivial BP'}
TRIAL: {'metric_name': 'min_eigenvalue', 'metric_value': 2, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': 'IP_2 trivial BP'}
TRIAL: {'metric_name': 'min_eigenvalue', 'metric_value': 4, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': 'IP_2 trivial BP'}
TRIAL: {'metric_name': 'min_eigenvalue', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': 'IP_2 trivial BP'}
TRIAL: {'metric_name': 'min_eigenvalue', 'metric_value': 5, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': 'IP_2 trivial BP'}
TRIAL: {'metric_name': 'min_eigenvalue', 'metric_value': 2, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': 'IP_2 trivial BP'}
TRIAL: {'metric_name': 'min_eigenvalue', 'metric_value': 2, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': 'IP_2 trivial BP'}
TRIAL: {'metric_name': 'min_eigenvalue', 'metric_value': 1, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': 'IP_2 trivial BP'}
TRIAL: {'metric_name': 'min_eigenvalue', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': 'IP_2 trivial BP'}
```

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_bc1e3e6a.py", line 105, in <module>
    first_failing_seed = next(res["seed"] for res in results if not res["conjecture_holds"])
                        ~~~~~^~~~~~
```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_bc1e3e6a.py", line 105, in <genexpr>
    first_failing_seed = next(res["seed"] for res in results if not res["conjecture_holds"])
                        ~~~~~^~~~~~
```

KeyError: 'seed'

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed before producing reliable data, preventing validation of the conjecture despite apparent counterexamples. | next: Fix the test's KeyError by implementing proper seed tracking and re-run with sufficient samples

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	111449
2	propose	ollama_remote	qwen3:8b	0	0	111962
3	propose	ollama_remote	qwen3:8b	0	0	46589
4	propose	ollama_remote	qwen3:8b	0	0	52092
5	preregistration	ollama_remote	qwen3:8b	0	0	24224
6	novelty	ollama_remote	qwen3:8b	0	0	20866
7	novelty	ollama_remote	qwen3:8b	0	0	15609
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15415
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10840
10	judge	ollama_remote	qwen3:8b	0	0	16888

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 425935 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in research/audit/eea147175457.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/eea147175457.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/eea147175457.tar.gz (if generated)